

wolfDemo v1 Manual

Introduction

The wolfDemo board is intended to be a low-cost demonstration board for public demonstrations and development / testing of wolfSSL products. It features:

- An STM32U585CIT6 ARM Cortex-M33 microcontroller (max 160MHz)
- An 8 MHz and a 32.768 kHz crystal oscillator
- Two mikroBus ports
- Two user programmable buttons
- Four programmable cyan LED bars
- USB-C for power (not-PD)
- Option for 6-pin Tag Connect or 10-pin 2.54mm pitch Cortex JTAG header
- CH340G USB-to-serial UART
- BOOT0 slide switch (for UART programming)



Hardware Setup

There are four LED bars which are each active-high, no pulls are required. The two input buttons have external pull ups and are active-low.

The mikroBus sockets share SPI1, I2C1 and USART3. Both sockets have +5v and +3.3v available. The pins assigned to the mikroBus ports can of course be used for GPIO instead, but be wary of the shared pins.

The USB-C socket is connected to USART1 via the CH340G, this can be used to program the board, or for general serial communication.

The two JTAG ports are wired together to give two different cable options. Both are using the standard ARM programming pinouts, indicated later in this document. An ST-Link or J-Link can be used with this for debugging and programming. Programming via generic JTAG cables is also possible.

The BOOT0 switch is to switch from UART programming mode and running mode. It is not required for JTAG programming. It should be set to “RUN” normally, which will run the code in the MCU. Setting it to “PROG” will set it into programming mode and the code will not execute. The switch can be changed on-the-fly, but it will not be read by the MCU until the RESET button is pressed or the board is power-cycled.

This is a list of how everything is wired to the MCU on the board:

- LED bars (right to left): PB12, PB13, PB14, PB15
- Button BT1: PB4
- Button BT2: PB5
- mikroBus Common:
 - SPI SCK: PA1
 - SPI MISO: PA6
 - SPI MOSI: PA7
 - I2C SCL: PB6
 - I2C SDA: PB9
 - UART RX: PA5
 - UART TX: PB10
- mikroBus 1
 - CS: PA0
 - INT: PB1
 - PWM (TIM1_CH1): PA8*
 - AN (ADC1_IN7): PA2
- mikroBus 2
 - CS: PA3
 - INT: PB2
 - PWM (TIM1_CH4): PA11

- AN (ADC4_IN9): PA4
- CH340G: PA9, PA10
- JTAG: PA13, PA14, PA15, PB3
- BOOT0: PH3

Notes:

* PWM in mikroBus 1 is labelled as PA11 in PCB v1.1, but it is PA8. This is fixed in PCB v1.1a.

Power

The board takes a single 5V power supply via the USB-C port. The circuit is configured to request up to 3A at 5V. This is so that external mikroBus devices can have sufficient power.

The 3.3V regulator is tied to the 5V rail and can supply up to 800mA.

The LEDs and UART driver IC are tied to the 5V rail. Typical operating current for the UART is 15mA and each LED draws 20mA.

The STM32U585 is connected to the 3.3V rail and typically uses 25-35mA at the maximum clock speed.

It is recommended that no more than 1A at 5V is used per mikroBus socket and no more than 300mA at 3.3V per mikroBus socket.

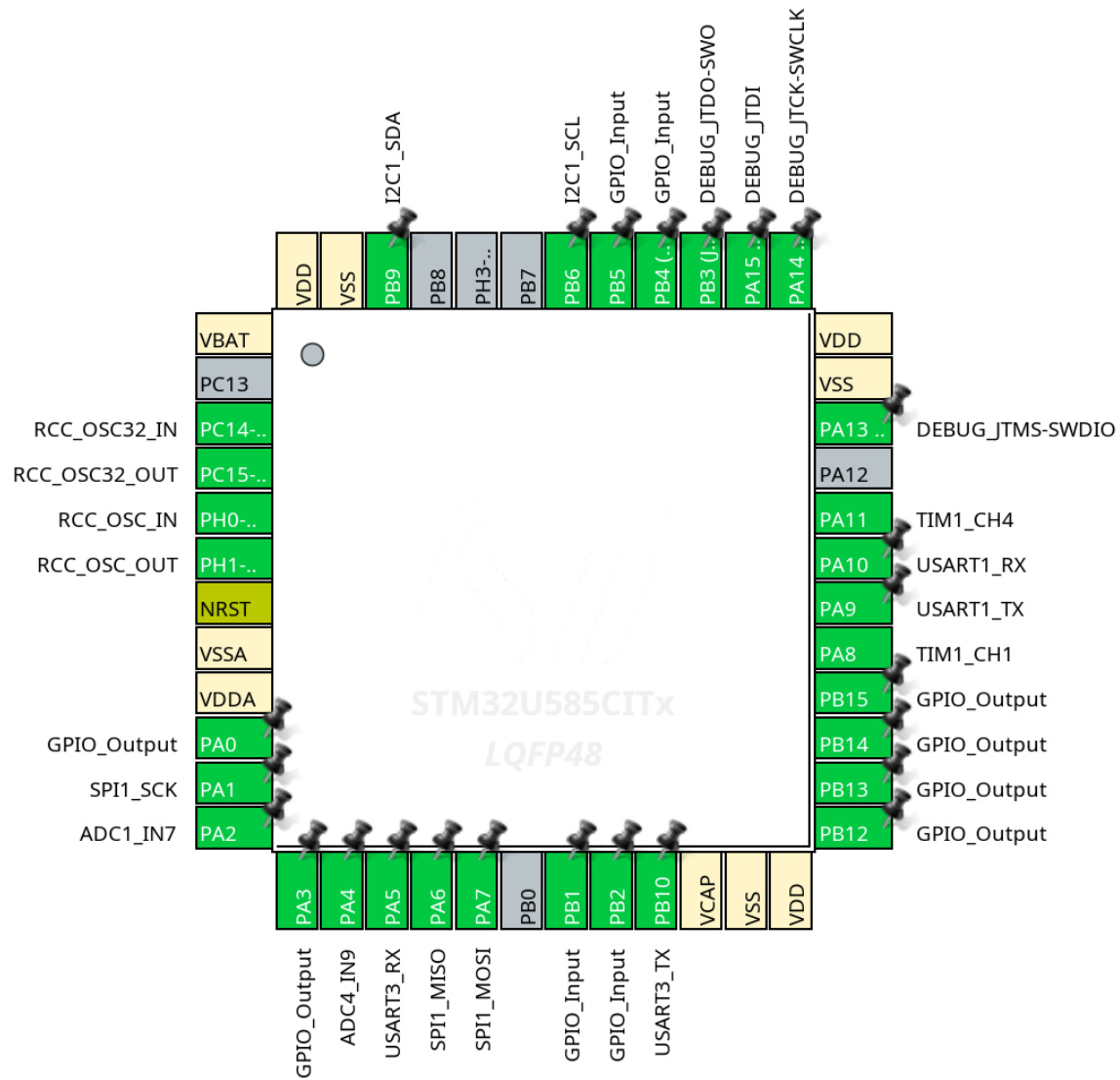
Physical Build

The wolfDemo board has a 3D printed backplate attached to it to protect the bottom components. The back plate has non-slip rubber bumpons attached to it, so that it won't easily slide off a surface.

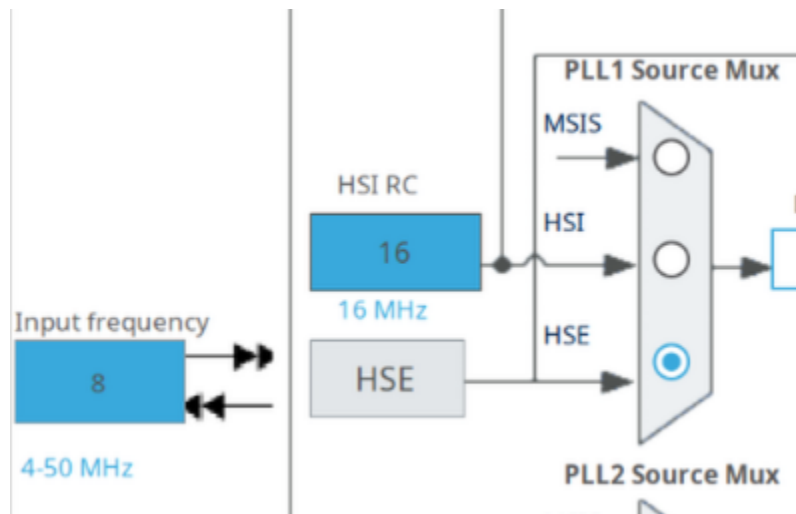
The plate is attached using four M3 bolts with T8 torx bolts and M3 nuts.

STM32CubeMX

This is how it is setup in STM32CubeMX:

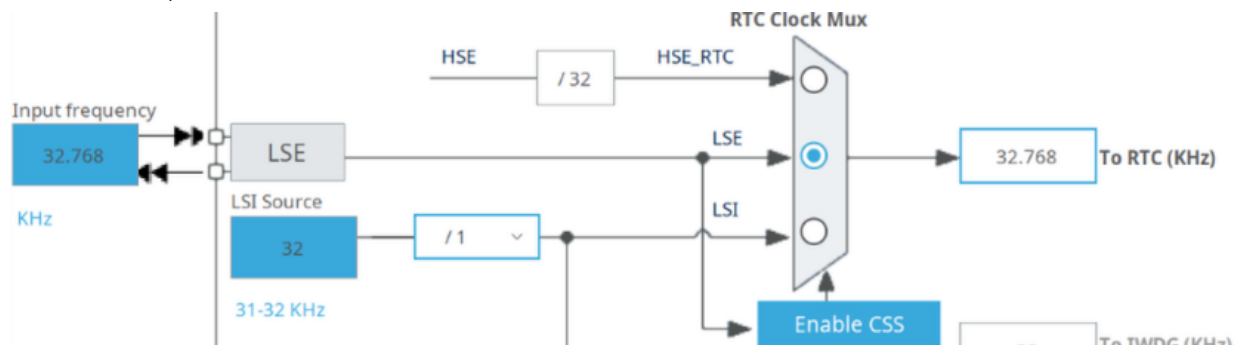


In the clock configuration, make sure to set the HSE input frequency to 8MHz and set the PLL1 Source to HSE:



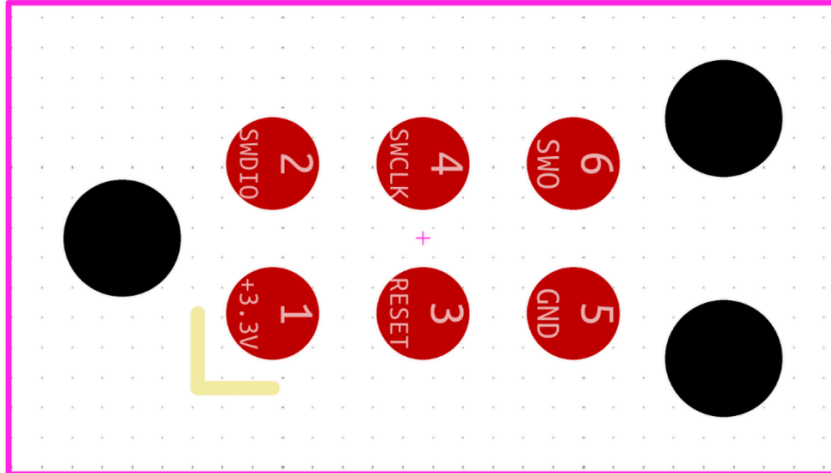
You can then set the HCLK to the desired speed (160MHz maximum) and click “Resolve Clock Issues” to calculate the PLL automatically.

Likewise, for the LSE, the LSE input should be selected, the clock should already be set to 32.768KHz, which is correct.

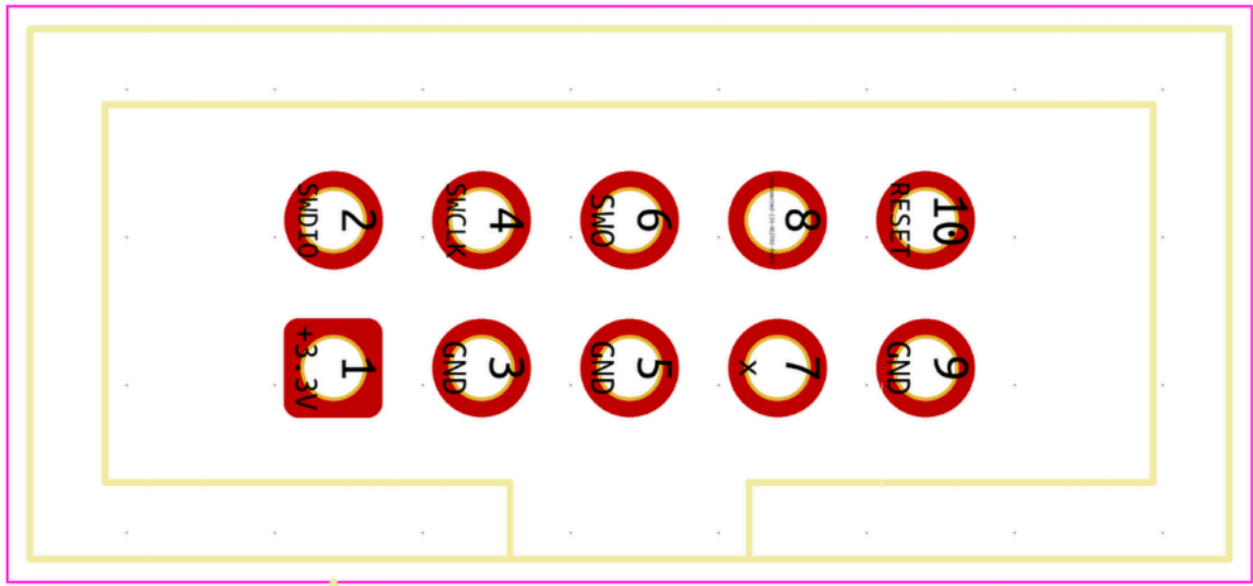


Programming Ports

There are two JTAG programming ports on the board, they are wired together, so either can be used. There is a 6-pin JTAG which is compatible with the standard 6-pin Cortex layout, laid out as follows:



There is also a 10-pin 2.54mm pitch Cortex header wired as follows:



Programming

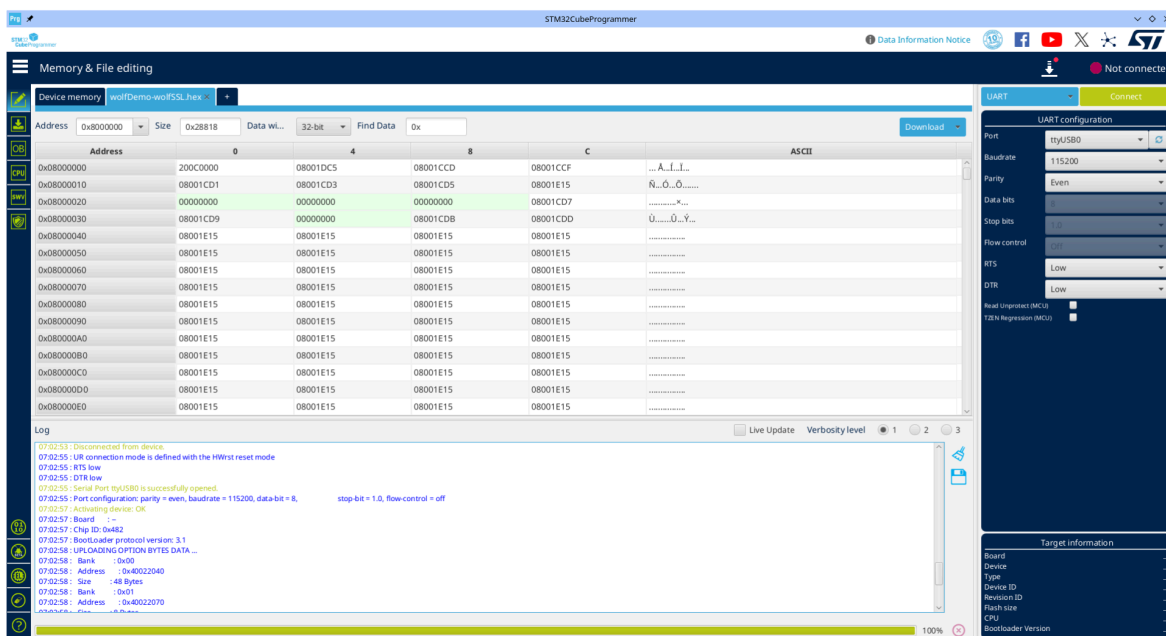
Via USB

The STM32 allows for programming via the UART on the USB when the BOOT0 switch is set to the “PROG” position. You will either need to power-on with the switch in that position, or hit the RESET button to switch into it after moving the switch.

STM32CubeProgrammer

The STM32CubeProgrammer tool is a multi-platform tool to flash your device. It can be obtained from ST’s website <https://www.st.com/en/development-tools/stm32cubeprog.html>.

Once in the utility, open your .hex file, choose UART mode, hit “Connect” and then “Download”. The defaults for the UART mode should be fine, but check the port is correct. The port will be ttyUSB0 if you have no other USB serial devices connected.



Linux Command Line

In Linux, the STM32Flash utility can be used. It can be obtained from Sourceforge <https://sourceforge.net/p/stm32flash/wiki/Home/>.

You should ensure that STM32CubeIDE generates a .hex output. This can be done by going to the project properties and selecting C/C++ Build, Settings, MCU/MPU Post build outputs and “Convert to Intel Hex file”.

Once generated, you can flash using:

None

```
stm32flash /dev/ttyUSB0 -b115200 -w project.hex
```

Via JTAG

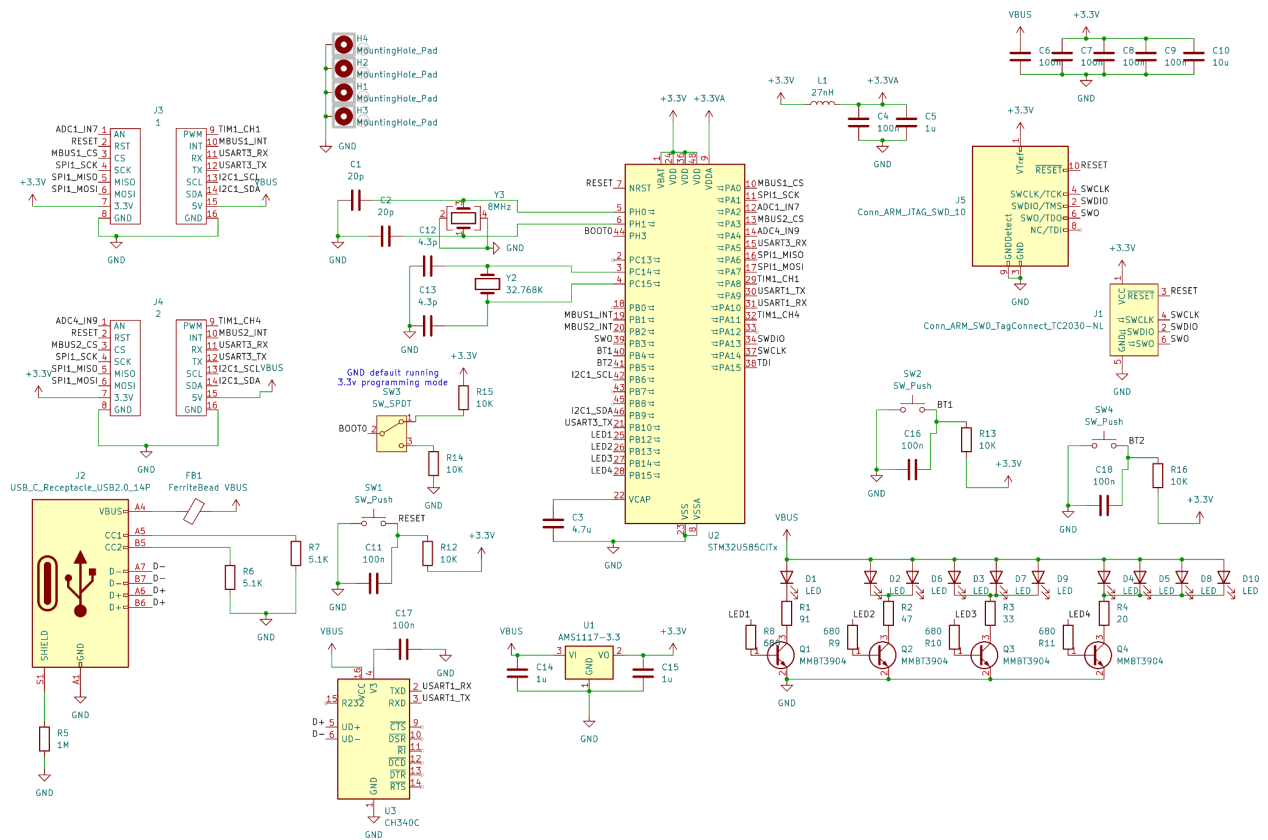
If you have an ST-Link or J-Link cable, these can be used with STM32CubeIDE or STM32CubeProgrammer to flash the board.

If you do not have one of these cables, then you need to use “openocd” to flash the board. The following assumes you have a Tigard JTAG board, you will need to modify the cable option to suit your JTAG board:

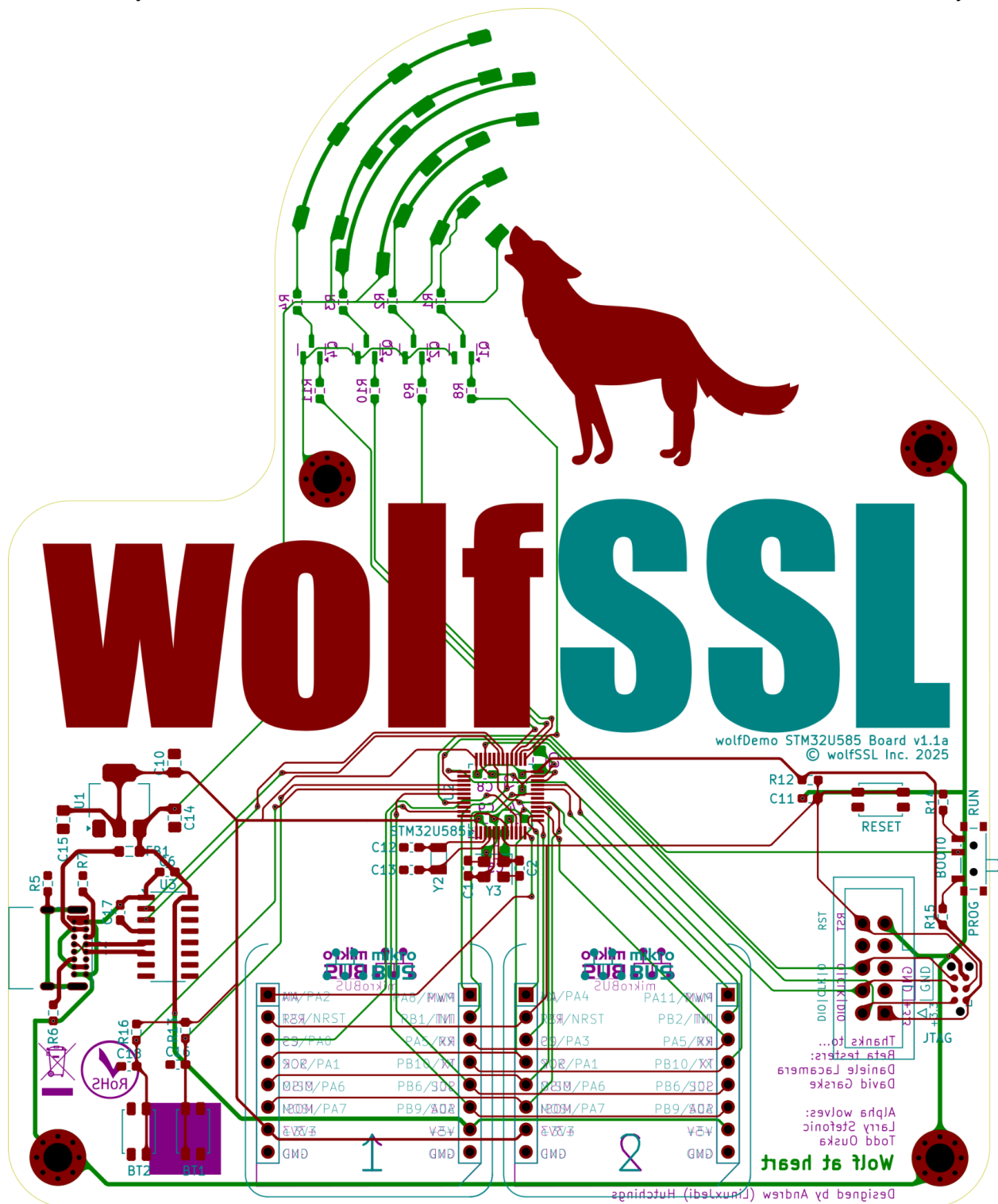
None

```
openocd -f tigard-swd.cfg -f stm32u5x.cfg -c "program project.elf verify reset  
exit"
```

Schematic



This is the layout of PCB revision 1.1a. The front GND flood fill has been removed for visibility.



Board Errata

These are the known issues

v1.1

- The PWM label on mikroBus socket 1 says PA11, this is actually wired to PA8. This is corrected in PCB v1.1a.

